Capítulo 6 · Seguridad en MongoDB

Manual de teoría — explicar antes de configurar

Destinatarios: personas que empiezan a administrar bases de datos y no han configurado usuarios, roles ni certificados.
Tecnología de referencia: MongoDB Community 8.0 con servicios locales de Docker.
Meta: comprender por qué un servidor que responde no es necesariamente un servidor seguro; distinguir red, identidad, permisos y cifrado; y razonar qué prueba demuestra cada protección.

La seguridad es una cadena de decisiones. Este manual comienza por el problema de negocio y avanza hacia la configuración; la práctica acompaña cada idea con observaciones de un servidor real.

---

1. Una base de datos puede funcionar perfectamente y ser insegura

Imagina que la librería guarda ventas, direcciones de envío y consultas de clientes. El equipo de atención puede consultar compras para resolver incidencias. Contabilidad necesita informes, pero no debería cambiar los datos históricos sin autorización. Un visitante anónimo no debería leer las ventas, aunque sepa a qué ordenador conectarse.

La base de datos podría responder en milisegundos y, sin embargo, estar abierta a cualquier persona que llegue a su puerto. **Disponibilidad, rendimiento y seguridad no son lo mismo.**

Cuatro preguntas distintas

La seguridad comienza respondiendo:

1. **¿Quién puede conectarse al servidor por la red?** Es la exposición de red.
2. **¿Quién es realmente el usuario que se conecta?** Es la autenticación.
3. **¿Qué operaciones está autorizado a realizar?** Es la autorización.
4. **¿Cómo protegemos los datos cuando viajan y cuando se almacenan?** Es el cifrado, más otras medidas de protección.

Que una capa esté correcta no implica que las demás también lo estén. Un servidor protegido con contraseña puede publicar innecesariamente su puerto a internet. Un usuario autenticado puede tener permisos excesivos. Una conexión con TLS puede proteger la comunicación, pero no convertir en segura una contraseña almacenada en un archivo público.

Un ejemplo de roles cotidianos

Pensemos en tres personas:

- **Administrador:** puede crear usuarios, configurar el servidor y gestionar datos.
- **Lector de atención al cliente:** necesita consultar ventas, pero no cambiarlas.
- **Editor autorizado:** puede consultar y modificar determinadas colecciones.

La aplicación debe conectar con la identidad apropiada para su función. Utilizar siempre una cuenta administradora simplifica las pruebas iniciales, pero anula la protección que queremos enseñar.

El principio del mínimo privilegio

Significa asignar **solo los permisos necesarios** para la tarea. Si una aplicación lee ventas, no necesita borrar bases de datos ni crear usuarios. Si alguien necesita corregir pedidos, eso no implica que pueda alterar la configuración de seguridad.

No es solo una idea moral: limita daños de errores de programación, contraseñas filtradas y operaciones equivocadas.

Pregunta para pensar: si un usuario puede iniciar sesión con su contraseña correcta y después recibe «Unauthorized» al intentar borrar ventas, ¿ha fallado la autenticación? No; podría haber funcionado la identidad y haber fallado correctamente una operación para la que no dispone de permiso.

---

2. La red: llegar al servidor no significa estar autorizado

Un programa cliente necesita localizar a MongoDB mediante una dirección y un puerto. En una práctica local podemos usar:

```
127.0.0.1:27017
```

`127.0.0.1` es la dirección de loopback: representa el propio ordenador desde el que se ejecuta el cliente. El puerto `27017` es el puerto de MongoDB empleado por nuestro servicio local de índices.

El laboratorio usa tres servicios de Docker:

Servicio	Puerto del equipo	Propósito
`mongo-indices`	`127.0.0.1:27017`	Datos ficticios; no exige login
`mongo-seguridad`	`127.0.0.1:27018`	Pruebas SCRAM y roles
`mongo-tls`	`127.0.0.1:27019`	Comunicaciones TLS y certificados

Los puertos externos no son los mismos que los puertos internos de los contenedores: Docker traduce un puerto del host al puerto de MongoDB dentro de cada servicio.

¿Por qué no publicar en todas las interfaces?

Cuando un servicio se publica en `0.0.0.0`, puede escuchar conexiones desde interfaces accesibles fuera del propio equipo, según cortafuegos y redes. Publicarlo en `127.0.0.1` reduce esa exposición desde el host. **No sustituye autenticación ni significa que todos los contenedores de la red Docker queden aislados unos de otros.**

El laboratorio de índices prescinde de autenticación para facilitar el primer aprendizaje, pero utiliza únicamente ventas falsas y publica su puerto en loopback. Ese diseño **no debe trasladarse como receta** a una base de datos real con información sensible.

Otros controles de red

En un despliegue de producción también se estudian firewalls, redes privadas, listas de acceso, segmentación y políticas de salida. Para empezar basta con entender que una contraseña es una capa distinta de la ruta de red.

---

3. Autenticación: demostrar quién eres

En la vida diaria puedes presentar una tarjeta de acceso o identificarte con usuario y contraseña. En MongoDB el cliente también debe aportar una identidad admitida por el servidor cuando el control de acceso está habilitado.

Autenticación responde a «¿Quién eres?» y **autorización** responde a «¿Qué puedes hacer?». Son decisiones consecutivas y diferentes.

SCRAM: el mecanismo habitual con contraseña

SCRAM significa *Salted Challenge Response Authentication Mechanism*. En la práctica el usuario conoce su contraseña, pero el servidor no necesita aceptar una contraseña enviada sin protección como único mecanismo de verificación. Interviene un intercambio de desafío/respuesta con protección criptográfica y valores específicos por usuario.

Para un principiante lo esencial es lo siguiente:

- El usuario posee un nombre de cuenta.
- Esa cuenta está definida en una base de datos de autenticación.
- La contraseña debe ser correcta.
- El cliente debe indicar o determinar en qué base de datos se autentica.

No es necesario memorizar los algoritmos criptográficos para entender el comportamiento, pero sí distinguirlos de la seguridad del canal: **SCRAM y TLS cumplen funciones diferentes**.

El parámetro authSource

Imagina que el administrador fue creado en la base `admin`. Eso no significa que todas sus operaciones sean sobre `admin`; es simplemente el lugar en que se verifica su identidad. Después puede recibir permisos sobre otras bases.

Un comando típico de cliente es:

```
mongosh --host localhost --port 27018 --username adminlab --authenticationDatabase admin --password
```

La opción `--password` sin contraseña escrita en la línea solicita introducirla sin dejarla como texto en el historial del terminal. Esta línea se usa en un entorno que exponga dicho puerto. En Docker podemos entrar a `mongosh` directamente dentro del contenedor, como explican las prácticas.

Si se indica una base de autenticación equivocada, el servidor puede rechazar las credenciales aunque la contraseña memorizada sea correcta. Por eso «error de login» no significa siempre «contraseña tecleada mal».

No reutilizar contraseñas de demostración

El repositorio incluye una plantilla `.env.example`, pero el archivo privado `.env` debe usar valores específicos de laboratorio. Una contraseña de clase no es una contraseña admisible para una organización. No debe aparecer en capturas de pantalla, commits ni informes de errores.

---

4. Autorización: quién puede leer, insertar, actualizar o borrar

Una vez identificada la persona, MongoDB comprueba si tiene privilegios sobre el recurso y la acción. Esto se organiza habitualmente mediante **RBAC**, es decir, control de acceso basado en roles.

Qué contiene un rol

Un rol representa un conjunto de permisos. En el caso más simple usaremos los roles incorporados:

- `read`: autoriza operaciones de lectura dentro del alcance de la base indicada.
- `readWrite`: añade operaciones de modificación de datos dentro de ese ámbito.
- `root`: concede privilegios de administración amplios; no debe asignarse a una cuenta usada por un alumno para leer un documento.

Los roles no tienen «poder mágico». Su alcance depende de la base a la que se asocian y de las operaciones exactas que incluyan.

Qué significa que una operación sea rechazada

Supongamos que `lectorlab` puede ejecutar:

```
db.secretos.findOne()
```

pero al intentar:

```
db.secretos.insertOne({texto: "algo nuevo"})
```

obtiene un error de autorización. **Ese segundo resultado indica que el control de acceso está funcionando**, siempre que el usuario sea realmente el lector esperado y no una cuenta administradora.

El riesgo de probar con root

Si para una actividad de permisos te conectas como root y todo funciona, no has probado que un lector esté limitado. La comprobación correcta debe realizarse **con la identidad sometida a prueba**.

Privilegio, rol, identidad: tres conceptos diferentes

Una **identidad** designa la cuenta. Un **privilegio** es el permiso de realizar una acción sobre un recurso. Un **rol** agrupa privilegios y se asigna a identidades. Podemos crear roles propios para casos muy precisos, pero el curso comienza con `read` y `readWrite`, que permiten observar la diferencia sin complejidad innecesaria.

5. Una demostración completa de SCRAM y RBAC

Este esquema resume tres comportamientos distintos:

```
Cliente SIN credenciales
    │
    └── intenta leer curso_seguridad.secretos
            └── RECHAZADO: falta autenticación

Cliente con lectorlab + contraseña correcta
    │
    ├── leer un secreto ──────────────── PERMITIDO
    │
    └── insertar nuevo secreto ──────── RECHAZADO: no tiene permiso

Cliente con escritorlab + contraseña correcta
    │
    └── insertar un documento ────────── PERMITIDO
```

Es posible que las dos denegaciones se parezcan superficialmente. Pero **tienen causas diferentes**:

- El visitante no dispone de una identidad autenticada.
- El lector posee una identidad válida, pero no la autorización para insertar.

Cómo interpretar los mensajes

«Authentication failed» apunta al proceso de identificación o al contexto de autenticación. «not authorized» indica que la operación requerida no está permitida para la identidad actual. Son pistas para analizar, no pruebas completas por sí mismas: se deben acompañar de la cuenta y el comando que se intentó ejecutar.

Por qué necesitamos una prueba positiva y otra negativa

Si solo comprobamos que `lectorlab` consigue leer, no sabemos todavía si también puede borrar. Y si solo comprobamos que una escritura falla, quizá se debió a una contraseña equivocada o a estar conectados al servidor incorrecto.

La secuencia correcta comprueba la identidad, una operación permitida y otra denegada. La práctica de seguridad ejecuta justamente estas tres situaciones.

---

6. TLS: proteger la comunicación entre cliente y servidor

La librería trabaja con información que puede ser sensible. Cuando un cliente envía consultas y recibe resultados, los datos atraviesan un canal de comunicación. TLS (*Transport Layer Security*) busca proteger ese intercambio frente a lectura y manipulación en tránsito.

El candado y la identidad del servidor

A menudo se habla de TLS como «un canal cifrado». Eso es cierto pero incompleto. También necesitamos **comprobar con quién hablamos**. Un certificado de servidor permite que el cliente valide aspectos de identidad y confianza, incluidos el nombre del servidor y la cadena de firmas, en función de su configuración.

Un flujo simplificado:

```
1. Cliente pide conexión TLS
2. Servidor presenta su certificado
3. Cliente valida la firma, la cadena de confianza y el nombre esperado
4. Se establece un canal protegido
5. Cliente se autentica para operar con la base si así se exige
```

TLS no asigna permisos sobre colecciones. Un canal cifrado puede transportar una petición de lectura que el servidor después deniega. También puede haber una conexión TLS válida con un usuario sin privilegios suficientes.

¿Qué es una CA?

Una autoridad certificadora (CA) firma certificados. En clase fabricaremos una CA de demostración y certificados temporales para poder observar el proceso sin comprar servicios. Los certificados de clase no sirven para confiar en identidades reales de internet ni para desplegar una infraestructura empresarial.

Prueba negativa deliberada

En una instancia configurada con `requireTLS`, intentamos conectarnos sin TLS. La conexión debe fallar. Después conectamos con TLS y validación de la CA. Esa segunda conexión debe llegar al servidor.

La prueba no consiste en que «al final se conecte». Consiste en observar que **un tipo de comunicación se rechaza y otro se permite según la política configurada**.

Riesgos frecuentes

Desactivar la validación de certificados para «arreglar» un error elimina una parte importante de la protección. Usar certificados caducados, con nombres distintos o firmados por una CA no confiable también causa errores. Es preferible interpretar cada mensaje a introducir opciones inseguras sin comprenderlas.

---

7. X.509: usar un certificado para autenticar a un cliente

TLS y X.509 están relacionados, pero debemos enseñar qué representa cada cosa.

- **TLS** protege el canal de comunicación y permite validar al servidor.
- **X.509 para autenticación de cliente** utiliza un certificado como parte de la identidad que MongoDB verifica.

MongoDB Community 8.0 admite autenticación X.509. No hace falta Enterprise para el ejemplo de clase.

Una cuenta externa

En MongoDB, un usuario autenticado por certificado se registra mediante una identidad basada en el sujeto del certificado, generalmente en la base especial `$external`. Por ejemplo, una identidad de laboratorio puede contener un nombre distinguido como:

```
CN=alumno-x509,OU=Alumnos,O=MongoDBFormacion
```

Ese identificador debe concordar con la representación que el servidor espera. No es un alias elegido libremente en el momento de conectarse.

El certificado no es un rol

Un certificado válido puede permitir demostrar «soy quien afirma ser». Pero las operaciones posibles dependen de los roles que el servidor haya asociado a esa identidad. En nuestra demostración, el usuario X.509 puede **leer** una colección y **no puede escribir** en ella.

Esta distinción evita una confusión común: creer que al disponer de un certificado toda la base queda abierta.

Condiciones para que funcione

Necesitamos TLS, una CA confiable, certificados válidos, la identidad registrada, un cliente que presente su certificado y permisos adecuados. Un error en cualquiera de estas piezas puede causar que el acceso falle. Por esa razón X.509 se estudia después de comprender SCRAM y roles, no como primer contacto con seguridad.

---

8. LDAP y otros directorios de identidad

Una organización grande puede querer gestionar identidades desde un directorio central, de forma que las cuentas de empleados no se creen por separado en cada servidor. LDAP es un protocolo tradicional asociado a ese escenario.

Por qué aparece en un temario de seguridad

Históricamente MongoDB Enterprise permitió utilizar mecanismos relacionados con LDAP para autenticación o autorización contra servicios de directorio. Entender ese modelo ayuda a reconocer qué problema empresarial intenta resolver: **centralizar identidades y grupos**.

Lo que no debemos prometer en MongoDB Community

La integración LDAP autogestionada pertenece a capacidades de Enterprise y **está obsoleta a partir de MongoDB 8.0**. No podemos reproducirla honestamente activando una opción inexistente en nuestro contenedor Community. El curso la explica conceptualmente, pero no finge haber realizado un laboratorio real de LDAP.

Qué otros enfoques existen

Las organizaciones pueden estudiar identidades federadas y opciones basadas en estándares como OpenID Connect en entornos compatibles. Cada opción depende de la edición y del despliegue. Para un principiante la idea principal es diferenciar:

- cuenta local en la base;
- cuenta gestionada por un directorio;
- autenticación basada en certificado;
- permisos que MongoDB aplica después de conocer la identidad.

---

9. Cifrado en reposo: proteger archivos guardados

TLS protege datos **mientras viajan**. ¿Qué ocurre si alguien consigue acceso indebido a un disco o una copia de los archivos del servidor? Ese es otro problema: **protección de datos en reposo**.

Tres capas distintas

1. **Cifrado del disco o volumen:** lo ofrece, por ejemplo, el sistema operativo o una infraestructura de almacenamiento. Puede proteger ante ciertas amenazas físicas, según gestión de claves y condiciones de montaje.
2. **Cifrado nativo del motor MongoDB:** en instalaciones autogestionadas, es una capacidad asociada a MongoDB Enterprise.
3. **Cifrado del lado del cliente o de campos:** determinadas soluciones protegen campos antes de almacenarlos; requieren su propio diseño de claves y funcionamiento.

No son equivalentes. Activar cifrado de disco en un portátil **no demuestra** que MongoDB Community implemente por ello el cifrado nativo en reposo del motor.

La gestión de claves es parte de la seguridad

Un sistema que cifra archivos pero guarda la clave junto a los datos sin protección adecuada ofrece una defensa muy limitada frente a ciertas amenazas. La gestión de claves, acceso, rotación, respaldo y recuperación debe formar parte del diseño.

Qué podrá hacer el alumno en este curso

Identificar qué datos se quieren proteger, el tipo de amenaza, qué capa de cifrado corresponde y qué tecnología o edición haría falta. **No se realizará una falsa práctica de cifrado nativo Enterprise utilizando un contenedor Community.**

---

10. Usuarios, contraseñas y secretos: el aspecto humano

Muchos incidentes ocurren porque una contraseña termina en un repositorio público, un fichero compartido o una captura de pantalla. La seguridad no empieza ni termina con el servidor; incluye cómo gestionamos sus credenciales.

Contraseñas en comandos

Es tentador escribir:

```
mongosh -u usuario -p contraseña-visible
```

No es recomendable para un entorno real: puede quedar en el historial de comandos, registros y herramientas de inspección. Es preferible usar mecanismos de solicitud interactiva, variables protegidas o gestores de secretos, según el entorno.

En clase el archivo `.env` contiene contraseñas desechables y no debe subirse al repositorio. El archivo `.env.example` es únicamente una **plantilla**; no debe incluir secretos válidos.

El principio de separar funciones

La cuenta que prepara usuarios no tiene que ser la cuenta de la aplicación. La cuenta que ejecuta informes no tiene que poder borrar colecciones. La cuenta que enseña una práctica no tiene que utilizar datos reales de clientes.

Un incidente sencillo para razonar

Un desarrollador publica por error una contraseña de demostración. ¿Qué debe hacer? Eliminarla del último commit no basta si ya quedó en el historial y puede seguir siendo válida. Debe considerarse comprometida y **rotarse**. En este curso usamos datos ficticios para reducir consecuencias, pero el hábito correcto debe aprenderse desde el principio.

---

11. Logs, errores, copias y recuperación

Cuando algo falla en una base de datos necesitamos información para diagnosticarlo. Los logs pueden mostrar arranque del servidor, problemas de conexión y otros eventos. Su contenido debe tratarse con cuidado para no revelar contraseñas ni datos sensibles.

Log no equivale a auditoría formal

Un log general de Docker o MongoDB ayuda a investigar incidentes, pero no equivale a una solución completa de auditoría regulatoria. Las capacidades de auditoría formal de MongoDB autogestionado dependen de la edición Enterprise y de una configuración apropiada.

Copias de seguridad

Cifrar la comunicación no sustituye tener copias. Tampoco una copia sin probar garantiza poder recuperar el servicio. Una organización debe definir qué se respalda, dónde se guarda, quién puede acceder, cómo se protege y cómo se verifica la restauración.

Qué haremos en el laboratorio

Cuando aparezca una conexión denegada, leeremos el error concreto, identificaremos qué capa falló y cambiaremos **solo la configuración necesaria**. No deshabilitaremos autenticación para que deje de mostrarse el error.

---

12. Método de diagnóstico: no quitar seguridad para «hacer que funcione»

Imagina que un alumno recibe `not authorized` al intentar insertar un documento. Una reacción habitual es intentar conectarse como `root`. Eso permitiría la operación, pero destruiría la enseñanza: dejaría de demostrar que el lector no puede escribir.

La secuencia de diagnóstico correcta es:

1. **Identificar servidor y puerto.** ¿Conectamos a la instancia de índices o a la de seguridad?
2. **Confirmar identidad.** ¿La contraseña y la base de autenticación corresponden al usuario previsto?
3. **Identificar la operación.** ¿Se intenta `find`, `insertOne`, `updateOne` o `deleteOne`?
4. **Revisar el rol.** ¿`read` debe permitir esa acción? No si implica insertar, actualizar o eliminar.
5. **Interpretar el resultado.** ¿El rechazo confirma la política o señala una configuración incorrecta?
6. **Registrar una explicación.** No basta con copiar el mensaje de error.

Caso final: cuatro conexiones

Situación	¿Qué esperarías?	Motivo
Anónimo lee datos privados	Rechazo	Falta identidad
Lector autenticado consulta	Éxito	Tiene privilegio de lectura
Lector autenticado inserta	Rechazo	No tiene escritura
Cliente sin TLS frente a `requireTLS`	Rechazo	No cumple protección del canal

Si puedes explicar por qué cambian estas respuestas, ya posees el fundamento para aprender configuraciones más avanzadas.

---

13. ¿Qué protege cada barrera? Un modelo de amenazas sencillo

Antes de decidir una medida de seguridad hay que preguntarse **de qué queremos defendernos**. Esa pregunta se llama análisis de amenazas. No hace falta ser especialista en ciberseguridad para realizar su versión básica.

Consideremos una base de datos de ventas con nombres, direcciones y operaciones comerciales.

- Una persona que comparte nuestra red podría intentar conectar al puerto de la base.
- Un empleado podría tener una contraseña legítima pero utilizarla para consultar información que no necesita.
- Un programa de la empresa podría cometer un error e intentar borrar documentos.
- Un intruso podría obtener una copia de almacenamiento o de una copia de seguridad.
- Alguien podría copiar un certificado o una contraseña a un repositorio público.

No hay una sola barrera que resuelva todos estos problemas. La limitación de red dificulta que ciertos clientes alcancen el servidor; la autenticación comprueba identidades; la autorización limita acciones; TLS protege la comunicación; el cifrado y la custodia de claves se dirigen a otras amenazas.

La regla de las evidencias separadas

Una prueba de `ping` no demuestra que la información esté protegida. Una conexión TLS no demuestra que una cuenta tenga privilegios mínimos. Un documento cifrado tampoco prueba que existan copias de seguridad utilizables.

Cuando evaluamos seguridad debemos enunciar una propiedad específica y diseñar una prueba que la observe. Por ejemplo: «La cuenta lectora no puede insertar datos en la colección de ventas». Para comprobarla, nos autenticamos con esa cuenta y ejecutamos un `insertOne`, buscando la denegación prevista.

Pregunta para transferir la idea

La base solo escucha en `127.0.0.1`. ¿Podemos quitar la autenticación porque el puerto es local? No: programas locales o procesos que ya disponen de acceso al equipo podrían intentar conectarse; además, un futuro cambio en la publicación de puertos ampliaría la exposición.

---

14. Qué es una URI de conexión y por qué puede fallar

Los clientes de MongoDB suelen recibir los datos de conexión mediante una URI. Una representación sencilla de un servidor local es:

```
mongodb://localhost:27018
```

La cadena incluye el esquema `mongodb://`, un host y un puerto. Sin embargo, en un servidor con autenticación faltan otros datos que pueden aportarse mediante opciones o un mecanismo seguro de configuración.

Usuario, contraseña y authSource

Un administrador definido en `admin` puede conectarse para operar después con otra base. Si la URI no indica correctamente la base de autenticación, el servidor puede no encontrar su identidad en el lugar esperado.

Ejemplo didáctico de forma —sin contraseña real—:

```
mongodb://adminlab@localhost:27018/?authSource=admin
```

En un programa real se usaría una forma de proporcionar el secreto que no lo exponga en el repositorio. La omisión deliberada de la contraseña en el ejemplo es parte de la enseñanza.

El error de «estoy en la base equivocada»

En `mongosh`, `use curso_seguridad` selecciona el contexto de operaciones posteriores. **No convierte automáticamente a un usuario autenticado en `admin` en una identidad distinta.** La base de autenticación y la base donde se ejecuta la consulta son conceptos diferentes.

El error de «el nombre del usuario basta»

Una cuenta llamada `lectorlab` puede estar definida en una base, mientras que otra cuenta del mismo nombre existe en otra base. Para interpretar una conexión debemos pensar en la identidad con su contexto de autenticación, no únicamente en la cadena de caracteres del nombre.

---

15. Qué ocurre cuando el permiso cambia mientras la sesión está abierta

Hasta ahora asignamos roles al crear usuarios y comprobamos operaciones. Pero en una empresa los privilegios pueden cambiar: una persona pasa de atención al cliente a otra función, deja el equipo o recibe una responsabilidad temporal.

Revocar permisos no es solo editar una etiqueta

La política de autorización debe reflejar que algunas acciones dejan de estar permitidas. MongoDB ofrece operaciones administrativas para conceder y revocar roles. Estas operaciones deben ejecutarse por identidades con privilegios suficientes y registrarse según las políticas del entorno.

La pregunta importante no es si el comando «funcionó», sino si **las operaciones efectivas** cambiaron para la cuenta afectada.

Cómo comprobar un cambio

Un experimento didáctico completo se organiza en tres observaciones:

1. Antes del cambio, confirmar el rol y una operación que está permitida.
2. Realizar el cambio de roles como administrador.
3. Reconectar cuando corresponda y repetir la operación para comprobar si su resultado concuerda con la nueva política.

No se debe concluir que una revocación está validada solo porque una orden administrativa devolvió éxito.

Precaución con cuentas de trabajo

No pruebes revocaciones en la cuenta administrativa del servidor. Utiliza usuarios desechables de laboratorio para no perder capacidad de preparar o recuperar el entorno.

---

16. Certificados: sujeto, emisor, validez y clave privada

Un certificado X.509 no es simplemente «un archivo que termina en .crt». Contiene datos estructurados que permiten relacionar una clave pública con cierta información de identidad, bajo una firma.

Cuatro piezas que conviene reconocer

Sujeto: a quién se refiere el certificado. Puede contener un nombre común, una organización u otros atributos. En la autenticación de clientes MongoDB puede utilizar una representación del sujeto como identidad.

Emisor: quién firmó el certificado. En un laboratorio puede ser nuestra propia autoridad certificadora de demostración.

Periodo de validez: fechas entre las que el certificado está previsto para ser válido. Una firma correcta no resuelve por sí sola el problema de un certificado caducado.

Clave privada: material secreto que no debe compartirse. El certificado público y la clave privada cumplen funciones distintas.

Leer un certificado

En una terminal con OpenSSL:

```
openssl x509 -in certs/client.crt -noout -subject -issuer -dates
```

El comando muestra atributos, pero no demuestra por sí mismo que MongoDB haya autenticado al cliente. Para eso necesitamos establecer una conexión y verificar los privilegios efectivos de esa identidad.

No desactivar validación por comodidad

Si el certificado no corresponde al nombre esperado o no se valida contra una CA confiable, eliminar la comprobación puede permitir una conexión, pero también renuncia a una protección importante. La respuesta correcta es diagnosticar la CA, los nombres y las fechas.

---

17. Exposición por error: secretos en el repositorio y en los logs

Un fallo frecuente consiste en utilizar contraseñas de laboratorio como si fueran secretos de producción. Los archivos de configuración, las capturas de pantalla y los mensajes de error pueden propagar información confidencial.

Archivos de ejemplo frente a archivos privados

`.env.example` es una plantilla para indicar qué variables necesita una aplicación. `.env` contiene los valores que se usarán en una instalación concreta. Aunque ambos tengan apariencia similar, **su tratamiento debe ser diferente**.

El archivo privado debe excluirse del control de versiones. Pero también conviene revisar que scripts y logs no impriman su contenido sin necesidad.

Qué hacer si una contraseña se publicó

Eliminar el texto del archivo visible no invalida una contraseña que alguien pudo copiar. La medida importante es considerarla comprometida y **rotarla**, además de revisar registros de acceso y alcance del incidente conforme al entorno.

Qué hacer si se publicó una clave privada

Una clave privada de servidor o cliente debe tratarse como secreto comprometido. La sustitución del certificado y las decisiones de revocación dependen de la infraestructura de confianza, no de borrar únicamente la copia visible.

Aprendizaje para un alumno

En un ejercicio siempre utilizaremos valores ficticios; eso reduce el daño potencial pero no justifica aprender hábitos que pondrían en peligro una instalación real.

---

18. Restauración y continuidad: la seguridad también incluye recuperación

Una base puede estar correctamente autenticada y cifrada, pero no ser útil si la empresa pierde sus datos por error, avería o corrupción. La seguridad y la continuidad del servicio se relacionan.

Preguntas que debe responder una estrategia de copia

- ¿Qué información se copia?
- ¿Cada cuánto se copia?
- ¿Quién tiene permiso para leer o restaurar la copia?
- ¿Cómo se protege el almacenamiento de las copias?
- ¿Se ha probado que la restauración funciona?

Una copia que nunca se restauró en un entorno de prueba es una promesa, no una recuperación demostrada.

Qué riesgo persiste con TLS y RBAC

TLS protege el tráfico, y RBAC restringe operaciones por cuenta. Ninguno evita por sí solo que un disco se rompa o que una cuenta autorizada borre datos accidentalmente. Por eso existen medidas de respaldo, procedimientos de recuperación y separación de privilegios.

Un caso de decisión

Si una cuenta de aplicaciones solo necesita escribir en una colección de pedidos, no debería poder restaurar o borrar la base completa. Son tareas diferentes que deberían asignarse a identidades y procesos distintos.

---

19. Cómo explicar una denegación sin recomendar privilegios excesivos

En un laboratorio el alumno recibe «not authorized» e intenta volver a ejecutar el comando como administrador. Esa acción puede hacer desaparecer el error, pero también impedir que entienda el control de acceso.

Método de análisis

Comienza por identificar la operación: no es lo mismo `findOne` que `insertOne`. Después identifica la identidad: ¿lector, escritor o administrador? Finalmente revisa el alcance del rol.

Ejemplo:

```
Cuenta: lectorlab
Base: curso_seguridad
Operación: insertOne en secretos
Rol: read sobre curso_seguridad
Resultado esperado: denegación
```

En este escenario **no hay un fallo que arreglar**. Hay una regla que funciona.

Cuándo un rechazo indica un problema

Si el lector intenta `findOne` sobre una colección que debería poder consultar y recibe una denegación, podría existir un error de rol, base de datos o identidad. Se investiga ese caso como un problema de configuración, no como una invitación a conceder `root`.

Una frase útil para enseñar

«Una autenticación exitosa no responde qué acciones están permitidas. Para saberlo hay que consultar la autorización en el contexto del recurso concreto».

---

20. Elegir una protección exige justificar sus límites

Resuelve estas cuatro situaciones antes de pasar a las prácticas:

Situación A. Un servicio publica MongoDB en todas las interfaces de una máquina conectada a una red compartida. ¿Qué capa revisarías primero? La exposición de red, sin olvidar autenticación.

Situación B. Un lector puede consultar documentos, pero intenta borrarlos. ¿Qué esperamos? Una denegación de autorización, siempre que su cuenta sea efectivamente de lectura.

Situación C. Una conexión TLS se establece correctamente, pero no se permite consultar una colección. ¿Es contradictorio? No: la protección del canal y los privilegios son independientes.

Situación D. Una empresa quiere proteger una copia robada del disco del servidor. ¿Basta TLS? No: hay que estudiar protección en reposo y custodia de claves.

No memorices «A = red, B = rol, C = TLS». Explica qué amenaza aborda cada control y qué amenaza **permanece sin resolver** cuando solo se aplica ese control.

Glosario de seguridad

Autenticación: verificación de identidad. **Autorización:** comprobación de privilegios para una acción. **RBAC:** modelo de permisos agrupados por roles. **SCRAM:** mecanismo de desafío/respuesta para autenticación con contraseña. **`authSource`:** base donde se verifica una cuenta. **TLS:** protección de comunicación en tránsito. **X.509:** estándar de certificados que puede emplearse para autenticar clientes. **CA:** autoridad que firma certificados. **LDAP:** protocolo de directorio, con integración autogestionada Enterprise obsoleta en MongoDB 8.0. **Cifrado en reposo:** protección de información almacenada. **Mínimo privilegio:** otorgar solo los permisos necesarios.

Fuentes oficiales

- Seguridad de MongoDB 8.0
- Autenticación en despliegues autogestionados
- SCRAM
- LDAP: obsolescencia en MongoDB 8.0
- Comparación Community / Enterprise

Ahora pasa al cuaderno de prácticas. Comenzaremos con una lectura anónima que tiene que fallar y avanzaremos hasta demostrar con otra identidad por qué la misma base permite leer pero rechaza escribir.